SUSE | NVIDIA

NVIDIA Run:ai on SLES15 RKE2

Proof of Concept Deployment and Validation Guide

Self-hosted platform backend, GPU node preparation, cluster attachment and demo workflow

Revision date: 2026-06-02

Document status: validated lab build, sanitized for sharing

Document information

Field	Value
Document type	Getting started / proof of concept guide
Primary audience	SUSE partner solution architects, platform administrators and POC reviewers
Environment	Single-node RKE2 lab with GPU Operator and NVIDIA Run:ai self-hosted backend on sles15 sp7
Security handling	Passwords, registration codes and API keys are redacted and replaced with placeholders

IMPORTANT	Do not store live passwords, SUSE registration codes, NGC API keys or platform administrator passwords in this guide. Use a password vault or a temporary environment variable and rotate any secrets that were previously copied into shared notes.

Contents

1. Purpose and scope
2. Architecture and lab topology
3. Prepare the host
4. Install the Kubernetes cluster
5. Install GPU support
6. Configure DNS, ingress and TLS
7. Configure registry access
8. Deploy the platform backend
9. Attach and validate the workload cluster
10. POC demo workflow
11. Operational checks and troubleshooting
12. Appendix: command reference

Purpose and scope

This guide documents a proof of concept deployment for NVIDIA Run:ai on a SUSE RKE2 cluster. It converts raw lab execution notes into a repeatable, SUSE-aligned technical guide that can be used to rebuild, validate and demonstrate the solution.

The deployment uses a single GPU node, a self-hosted NVIDIA Run:ai backend, the NVIDIA GPU Operator and local-path storage for lab persistence. The guide focuses on the POC flow and validation evidence rather than a production high availability design.

Expected outcomes

- The GPU node is registered to SUSE repositories and has correct system time.
- RKE2, kubectl and Helm are available on the node.
- The NVIDIA driver and GPU Operator expose GPU capacity to Kubernetes.
- The NVIDIA Run:ai backend is reachable through the configured FQDN and TLS certificate.
- The workload cluster is visible in the platform and can run GPU-backed demo workloads.

Writing and brand conventions used

- Sentence-style capitalization is used for section headings inside the document.
- Procedures use direct, active instructions and short command blocks.
- SUSE Pine and Jungle colors are used for headings, cover treatment, tables and callouts.
- Third-party trademarks are used only where needed for technical clarity.
- Credentials are not published in the body of the guide.

Architecture and lab topology

The POC uses a login node to reach a single GPU node. The GPU node hosts the SUSE operating system, RKE2 server, Kubernetes control plane components, the NVIDIA GPU Operator, monitoring components and the NVIDIA Run:ai self-hosted backend.

Component	Value used in the lab	Notes
Login node	172.16.0.90	Used for administrative access. Credentials are redacted.
GPU node	172.16.0.121	Hosts the RKE2 server and GPU workload stack.
Node DNS name	kube21.susepartner.com	Used as the platform FQDN in the lab.
Short host name	kube21	Used as a local alias in hosts file entries and certificate SANs.
Kubernetes distribution	SUSE RKE2	Installed as a single-node server for the POC.
Ingress class	nginx	Detected from the RKE2 ingress controller.
Storage class	local-path	Configured as the default storage class for the lab.

NOTE The IP addresses and host names in this guide reflect the POC lab. Replace them with site-specific values when repeating the deployment.

Prepare the host

Prepare the GPU node before installing Kubernetes and platform components. The preparation steps verify connectivity, operating system registration, repositories and system time.

Connect to the GPU node

Example command

```shell
ssh root@kube21.susepartner.com
```

Check the operating system

Verification command

```shell
cat /etc/os-release
```

Confirm that the host is running a supported SUSE Linux Enterprise release for the driver, Kubernetes and platform versions used in the POC.

Register the system

Use SUSEConnect to verify or complete product registration. Registration enables access to supported repositories and updates.

Registration and repository commands

```shell
SUSEConnect --status-text

sudo SUSEConnect -r <SUSE_REGISTRATION_CODE> -e <EMAIL_ADDRESS>

zypper repos
```

Synchronize system time

TLS, repository access and authentication flows are sensitive to clock drift. Enable NTP and verify the resulting time configuration.

Time synchronization commands

```shell
date
timedatectl

sudo timedatectl set-ntp true
sudo systemctl restart systemd-timesyncd
```

```
timedatectl status
date
```

Install the Kubernetes cluster

Install RKE2 and the administrative tools required to manage the cluster. The commands below match the lab build and can be adapted to use a different RKE2 version if required.

Install RKE2 server

Cluster installation commands

Shell

```
curl -sfL https://get.rke2.io | INSTALL_RKE2_VERSION=v1.33.5+rke2r1 sh -
systemctl enable --now rke2-server.service

mkdir -p ~/.kube
cat /etc/rancher/rke2/rke2.yaml > ~/.kube/config
```

Install kubectl and Helm

Administration tool commands

Shell

```
cd /usr/local/bin
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
chmod 700 kubectl
cd

curl -fsSL -o get_helm.sh
https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3
chmod 700 get_helm.sh
zypper install -y git-core
./get_helm.sh
```

Install Docker for registry login workflows

Docker installation commands

Shell

```
zypper in docker
systemctl start docker.service
systemctl status docker.service
systemctl enable docker.service
```

Configure a default storage class

Install the local-path provisioner and mark it as the default storage class for the lab. This is suitable for a single-node POC. Production deployments should use an enterprise-grade storage design.

Storage class commands

```shell
helm install local-path oci://dp.apps.rancher.io/charts/local-path-provisioner \
  --set global.imagePullSecrets={application-collection}

kubectl patch storageclass local-path \
  -p '{"metadata":
{"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'

kubectl get storageclass
```

Install GPU support

Install the NVIDIA driver packages and GPU Operator components so Kubernetes can advertise GPU resources to workloads.

Install driver packages

Driver installation commands

```shell
zypper ar https://developer.download.nvidia.com/compute/cuda/repos/sles15/x86_64/
cuda-sle15
zypper --gpg-auto-import-keys refresh

zypper install -y --auto-agree-with-licenses nv-prefer-signed-open-driver
version=$(rpm -qa --queryformat '%{VERSION}
' nv-prefer-signed-open-driver | \
  cut -d "_" -f1 | sort -u | tail -n 1)
zypper install -y --auto-agree-with-licenses nvidia-compute-utils-G06=${version}
```

Verify host driver readiness

Driver validation commands

```shell
lsmod | grep nvidia
cat /proc/driver/nvidia/version
```

Install the GPU Operator

The lab used the HelmChart custom resource below to install the GPU Operator from the NVIDIA Helm repository. The containerd socket and device plugin strategy match the RKE2 environment used in the POC.

GPU Operator HelmChart manifest

Shell

```
apiVersion: helm.cattle.io/v1
kind: HelmChart
metadata:
  name: gpu-operator
  namespace: kube-system
spec:
  repo: https://helm.ngc.nvidia.com/nvidia
  chart: gpu-operator
  version: v25.3.4
  targetNamespace: gpu-operator
  createNamespace: true
  valuesContent: |-
    toolkit:
      env:
      - name: CONTAINERD_SOCKET
        value: /run/k3s/containerd/containerd.sock
      - name: ACCEPT_NVIDIA_VISIBLE_DEVICES_ENVVAR_WHEN_UNPRIVILEGED
        value: "false"
      - name: ACCEPT_NVIDIA_VISIBLE_DEVICES_AS_VOLUME_MOUNTS
        value: "true"
    devicePlugin:
      env:
      - name: DEVICE_LIST_STRATEGY
        value: volume-mounts
```

Validate GPU visibility

GPU validation commands

Shell

```
kubectl get pods -n gpu-operator
kubectl describe node $(hostname) | grep -A5 -i "nvidia.com/gpu"
```

Configure DNS, ingress and TLS

Use a stable FQDN for the platform UI and API. The POC used local host-file resolution and a locally generated CA. For shared or production environments, use enterprise DNS and certificates issued by the approved internal or external certificate authority.

Set environment variables

Platform domain variables

```shell
export RUNAI_DOMAIN="kube21.susepartner.com"
export NODE_IP="172.16.0.121"
```

Resolve the FQDN locally

Node-side hosts file commands

```shell
grep kube21.susepartner.com /etc/hosts || \
  echo "172.16.0.121 kube21.susepartner.com kube21" >> /etc/hosts

getent hosts kube21.susepartner.com
hostname -f
```

On the laptop or jump host used to open the platform UI, add the same host entry or create a real DNS record.

Client-side connectivity commands

```shell
sudo sh -c 'echo "172.16.0.121 kube21.susepartner.com kube21" >> /etc/hosts'

ping -c 2 kube21.susepartner.com
curl -kI https://kube21.susepartner.com
```

Detect the ingress class

Ingress detection commands

```shell
kubectl get ingressclass
kubectl -n kube-system get pods | grep -i ingress

export INGRESS_CLASS="$(kubectl get ingressclass -o
jsonpath='{.items[0].metadata.name}')"
echo "$INGRESS_CLASS"
```

Create a local CA and certificate

Certificate generation commands

Shell

```
mkdir -p ~/runai-certs
cd ~/runai-certs
export RUNAI_DOMAIN="kube21.susepartner.com"

openssl genrsa -out runai-root-ca.key 4096
openssl req -x509 -new -nodes \
  -key runai-root-ca.key \
  -sha256 \
  -days 3650 \
  -out runai-root-ca.crt \
  -subj "/CN=runai-local-root-ca"

cat > runai-san.cnf <<EOF
[req]
default_bits = 2048
prompt = no
default_md = sha256
req_extensions = req_ext
distinguished_name = dn

[dn]
CN = ${RUNAI_DOMAIN}

[req_ext]
subjectAltName = @alt_names

[alt_names]
DNS.1 = ${RUNAI_DOMAIN}
DNS.2 = kube21
IP.1 = 172.16.0.121
EOF

openssl req -new -nodes \
  -out runai.csr \
  -newkey rsa:2048 \
  -keyout runai.key \
  -config runai-san.cnf

openssl x509 -req \
  -in runai.csr \
  -CA runai-root-ca.crt \
  -CAkey runai-root-ca.key \
  -CAcreateserial \
  -out runai.crt \
  -days 825 \
  -sha256 \
  -extfile runai-san.cnf \
  -extensions req_ext
```

Verify and create Kubernetes secrets

TLS secret commands

Shell

```
openssl x509 -in runai.crt -noout -subject -issuer -ext subjectAltName

kubectl create namespace runai-backend --dry-run=client -o yaml | kubectl apply -f -

kubectl -n runai-backend delete secret runai-backend-tls --ignore-not-found
kubectl -n runai-backend delete secret runai-ca-cert --ignore-not-found

kubectl create secret tls runai-backend-tls \
  -n runai-backend \
  --cert=~/runai-certs/runai.crt \
  --key=~/runai-certs/runai.key

kubectl -n runai-backend create secret generic runai-ca-cert \
  --from-file=runai-ca.pem=~/runai-certs/runai-root-ca.crt

kubectl -n runai-backend get secret runai-backend-tls
kubectl -n runai-backend get secret runai-ca-cert
```

Configure registry access

The platform backend pulls images and Helm charts from NVIDIA registries. Use an NGC API key with the required Run:ai entitlement.

Registry and Helm repository validation

```shell
export RUNAI_ADMIN_EMAIL="<ADMIN_EMAIL>"
export NGC_API_KEY="<NGC_API_KEY>"

docker login nvcr.io

curl -I -u '$oauthtoken':"$NGC_API_KEY" \
  https://helm.ngc.nvidia.com/nvidia/runai/index.yaml
```

HTTP result	Meaning
200	Access is working and Run:ai Helm repository access is available.
401	The key is incorrect or is not a valid NGC API key.
403	The key is valid, but the account does not have the required entitlement.

Create the image pull secret

Registry secret commands

```shell
kubectl create secret docker-registry runai-reg-creds \
  --docker-server=https://nvcr.io \
  --docker-username='$oauthtoken' \
  --docker-password="$NGC_API_KEY" \
  --docker-email="$RUNAI_ADMIN_EMAIL" \
  --namespace=runai-backend

kubectl -n runai-backend get secret runai-reg-creds
```

Deploy the platform backend

Deploy the self-hosted platform backend with Helm. The command sets the domain, ingress class, custom CA support and initial administrator credentials.

Backend deployment commands

Shell

```
export RUNAI_DOMAIN="kube21.susepartner.com"
export RUNAI_ADMIN_EMAIL="<ADMIN_EMAIL>"
export INGRESS_CLASS="$(kubectl get ingressclass -o
jsonpath='{.items[0].metadata.name}')"

read -s -p "Enter Run:ai UI admin password: " RUNAI_ADMIN_PASSWORD
echo

helm repo add runai https://helm.ngc.nvidia.com/nvidia/runai --force-update \
  --username='$oauthtoken' \
  --password="$NGC_API_KEY"

helm repo update

helm upgrade -i runai-backend -n runai-backend runai/control-plane \
  --set global.domain="$RUNAI_DOMAIN" \
  --set global.customCA.enabled=true \
  --set global.ingress.ingressClass="$INGRESS_CLASS" \
  --set tenantsManager.config.adminUsername="$RUNAI_ADMIN_EMAIL" \
  --set tenantsManager.config.adminPassword="$RUNAI_ADMIN_PASSWORD"
```

IMPORTANT	Enter the administrator password interactively. Do not store it in command history, tickets, chats or shared documents.

Attach and validate the workload cluster

After the backend is running, attach the RKE2 workload cluster from the platform UI or by using the installation command generated by the platform. The lab validation showed the cluster operators running in the runai namespace.

Cluster attachment checklist

1. Open the platform UI at https://kube21.susepartner.com.
2. Sign in with the initial administrator user configured during backend deployment.
3. Create or select the organization, department and project structure for the POC.
4. Add the RKE2 workload cluster using the platform-generated installation command.
5. Verify that the runai namespace contains the engine operator and platform operator pods.

Validation commands

Cluster validation commands

Shell

```
kubectl get pods -A
kubectl get pods -n runai
kubectl get pods -n runai-backend
kubectl get pods -n gpu-operator
kubectl get nodes -o wide
kubectl describe node $(hostname) | grep -A5 -i "nvidia.com/gpu"
```

Observed validation summary

Namespace	Observed status	POC interpretation
gpu-operator	GPU Operator, device plugin, DCGM exporter and validators were running or completed.	GPU enablement was successful.
kube-system	RKE2 control plane, Canal, CoreDNS, ingress-nginx, metrics server and snapshot components were running.	Core Kubernetes services were healthy.
monitoring	Prometheus, Alertmanager, kube-state-metrics and node exporter were running.	Monitoring stack was available for platform visibility.
runai-backend	Keycloak, PostgreSQL, Traefik and backend microservices were running. Some services had rollout restarts.	Backend became healthy after initial deployment restarts.
runai	engine-operator and runai-operator were running.	The workload cluster was attached and visible to the platform.

POC demo workflow

Use a small demo that shows platform value rather than only infrastructure installation. The suggested demo focuses on GPU visibility, controlled access, fractional GPU sharing and workload monitoring.

Demo story

A platform administrator provides a shared GPU cluster to a data science team. The team runs interactive and training workloads through the platform while the administrator controls quota and monitors utilization from a central UI.

Demo steps

Step	Action	Value to show
1	Show the cluster inventory and GPU capacity in the platform UI.	The platform can discover and govern the available GPU resources.
2	Create one department and at least one project for the POC team.	Projects provide logical tenancy and make quota assignment clear.
3	Create a compute resource with a fractional GPU request, for example 50 percent of a GPU.	Multiple workloads can share a GPU instead of leaving capacity idle.
4	Launch a short-running training or notebook workload using a demo image.	AI users can self-serve workloads through a governed interface.
5	Start a second fractional workload and show both workloads scheduled.	The scheduler improves GPU utilization on constrained lab hardware.
6	Open workload logs, events and metrics.	Operations teams get observability without direct node access.

Demo workload options

- **UI-first demo:** Use the Workload manager to create a training workload or workspace with the Run:ai demo image and a fractional GPU compute resource.
- **Kubernetes validation demo:** Run a simple pod with a GPU resource request to prove the device plugin exposes nvidia.com/gpu to the scheduler.
- **Operations demo:** Use the platform UI and kubectl together to show how platform state maps back to Kubernetes namespaces, pods and events.

Optional Kubernetes GPU smoke-test pod

Shell

```
apiVersion: v1
kind: Pod
metadata:
  name: gpu-smoke-test
spec:
  restartPolicy: Never
  containers:
  - name: cuda
    image: nvidia/cuda:12.4.1-base-ubuntu22.04
    command: ["/bin/bash", "-lc", "nvidia-smi && sleep 60"]
    resources:
```

```
    limits:
      nvidia.com/gpu: 1
```

Success criteria for the demo

- The platform shows the cluster as connected and healthy.
- GPU capacity is visible in both Kubernetes and the platform UI.
- At least one GPU workload starts successfully.
- The administrator can show project-level control or quota behavior.
- The UI shows workload state, logs or metrics that support an operations conversation.

Operational checks and troubleshooting

Use this section when a validation step fails or when the platform UI is not reachable.

Symptom	Check	Likely fix
UI does not open through the FQDN	Run get tenet hosts, ping and curl from the client machine.	Add the host entry on the client or create the DNS record. Verify ingress and TLS secrets.
Certificate warning or TLS mismatch	Inspect the certificate SAN values.	Regenerate the certificate with the FQDN, short name and node IP used by clients.
GPU resource not visible on the node	Check lsmod, /proc/driver/nvidia/version and GPU Operator pods.	Fix the driver installation, containerd socket value or GPU Operator rollout.
Helm repository returns 401	Validate the NGC API key value.	Generate or paste the correct NGC API key.
Helm repository returns 403	Check the account entitlement.	Use an account with Run:ai entitlement.
Backend pods restart repeatedly	Inspect kubectl logs and events in runai-backend.	Verify registry credentials, DNS, TLS secrets, storage and resource availability.
Workload remains pending	Describe the pod and check scheduling events.	Confirm GPU capacity, quota, node affinity, namespace access and image pull secrets.

Appendix: command reference

This appendix collects the primary commands in the order used during the lab. Replace placeholder values before running the commands.

Environment variables

```shell
export RUNAI_DOMAIN="kube21.susepartner.com"
export NODE_IP="172.16.0.121"
export RUNAI_ADMIN_EMAIL="<ADMIN_EMAIL>"
export NGC_API_KEY="<NGC_API_KEY>"
```

System registration

```shell
SUSEConnect --status-text
sudo SUSEConnect -r <SUSE_REGISTRATION_CODE> -e <EMAIL_ADDRESS>
zypper repos
```

RKE2 and tools

```shell
curl -sfL https://get.rke2.io | INSTALL_RKE2_VERSION=v1.33.5+rke2r1 sh -
systemctl enable --now rke2-server.service
mkdir -p ~/.kube
cat /etc/rancher/rke2/rke2.yaml > ~/.kube/config
kubectl get nodes
```

GPU Operator checks

```shell
kubectl get pods -n gpu-operator
kubectl describe node $(hostname) | grep -A5 -i "nvidia.com/gpu"
```

Backend checks

```shell
kubectl get pods -n runai-backend
kubectl get ingress -A
curl -kI https://kube21.susepartner.com
```

Cluster checks

```shell
kubectl get pods -n runai
kubectl get pods -A
kubectl get nodes -o wide
```

References

SUSE Documentation Style Guide:
https://documentation.suse.com/style/current/single-html/style-guide-adoc/style-guide-adoc.html

SUSE Brand Guide: https://brand.suse.com/

NVIDIA Run:ai self-hosted documentation: https://run-ai-docs.nvidia.com/self-hosted/

NVIDIA GPU Operator documentation: https://docs.nvidia.com/datacenter/cloud-native/gpu-operator/latest/